

COMMUNITY SECURITY ALERT SYSTEM (CSAS)

Felipe Jose Garzon Herrera Code: 20251020132
Henry Samuel Garrido Medina Code: 20251020125

Juan Esteban Quintero Gordillo Code: 20251020137
Gabriel Mateo Cusba Marin Code: 20251020128

1 | INTRODUCTION

University campuses frequently face security challenges that affect students, faculty members, and administrative personnel. Despite the existence of institutional security mechanisms, many incidents remain unreported due to the lack of accessible, real-time communication channels between the university community and security personnel.

Traditional reporting methods, such as phone calls, emails, or in-person reports, often result in delayed responses and fragmented information management. Furthermore, community members are rarely informed about nearby incidents as they occur, limiting their ability to take preventive actions.

To address these challenges, this project proposes the Community Security Alert System (CSAS), a collaborative platform designed to facilitate incident reporting, verification, and dissemination of real-time security alerts within a university environment.

The project was developed through four workshops covering problem analysis, requirements engineering, system design, risk analysis, and simulation-based validation.

4 | PROPOSED SOLUTION

The Community Security Alert System (CSAS) is a mobile-centered collaborative security platform designed to improve incident reporting efficiency and community awareness. The system combines crowd-sourced information, geolocation services, automated verification mechanisms, and emergency response support into a unified platform.

The primary objectives of CSAS are:

- Reduce reporting barriers.
- Improve response times.
- Increase community participation.
- Provide accurate situational awareness.
- Generate actionable security analytics.

5 | SIMULATION MODEL AND RESULTS

To validate the proposed architecture, a Discrete Event Simulation (DES) model was developed using Python and the SimPy framework.

The simulation reproduces the behavior of users interacting with the platform under realistic operating conditions.

The simulation was complemented by a graphical interface developed using Tkinter, allowing real-time observation of system events and incident processing workflows. The simulation demonstrated that the system can successfully process incidents, verify reports, and distribute alerts within acceptable response times.

DIMENSION	BASELINE	TARGET	EVIDENCE BASE	CONFIDENCE
Emergency Response Time Average time from incident report to alert delivery	~11.4 min Current response time using phone, email and manual coordination.	< 5 min Target response time established for CSAS platform (NFR-01).	Simulation optimization scenario achieved an average end-to-end response of 17.94 seconds .	HIGH Strong evidence from simulation validation.
Incident Recording Completeness Percentage of incidents that are captured and stored	~41% Many incidents go unreported through current fragmented channels.	> 85% Target completeness for reliable historical records (FR-07).	Prototype achieved 100% capture rate; production deployment requires mobile connectivity audit.	HIGH Conditional on connectivity validation.
Community Adoption Rate Percentage of students actively using the platform.	~12% Low adoption due to lack of awareness and reporting barriers.	≥ 60% Target adoption to sustain the reinforcing communication loop (FR-05).	Simulation validated at 67.62% adoption under optimized onboarding conditions (Workshop 4).	MEDIUM Depends on structured onboarding.
Alert Delivery Success Percentage of alerts successfully delivered to nearby users.	~87% Failures due to poor connectivity and lack of redundancy in channels.	> 99.9% Target delivery success rate (NFR-02).	NFR-02 validated in simulation. Twilio SMS fallback mechanism verified in prototype console.	HIGH Strong evidence from simulation and prototype.
Report Submission Friction Time and effort required for users to submit an incident report.	~3-5 min Phone calls or in-person reports take several minutes.	< 2 min Target submission time (NFR-06) using ≤ 3 interactions.	Prototype validation: average 35 seconds per report; NFR-06 confirmed using only two interactions.	VALIDATED Confirmed through prototype user testing.

2 | PROBLEM IDENTIFICATION

The initial phase of the project focused on identifying security-related problems experienced by university students.

A survey conducted among twenty students revealed that approximately 35% of respondents had experienced some form of security incident while on campus. Reported incidents included theft, harassment, and suspicious activities. Additionally, the majority of these incidents occurred during afternoon and nighttime periods, indicating a temporal concentration of security risks.

These findings suggest that existing reporting mechanisms are insufficient for ensuring timely communication and coordinated responses.

The analysis also identified several key challenges:

- Delayed incident reporting.
- Lack of real-time notifications.
- Fragmented information storage.
- Absence of community participation.
- Limited situational awareness.
- Lack of historical security analytics.

3 | DATA COLLECTION AND ANALYSIS

The project employed multiple data collection techniques to ensure a comprehensive understanding of campus security conditions.

Student Survey

Results indicated a generally positive attitude toward adopting a mobile-based collaborative security platform. Most participants expressed interest in receiving alerts and contributing incident information.

Existing security platforms operating in Bogotá were analyzed, including:

- Vigila App
- WhatsApp Community Networks
- SIURE UD

The benchmarking process identified strengths such as rapid communication and institutional integration, while also exposing weaknesses related to verification processes, data persistence, and real-time responsiveness.

METHOD	TOOL	SAMPLE	DUR.	OBJECTIVE
Interviews	Structured guide	5 security, 2 admin staff	1 week	Understand current workflows and identify unmet needs.
Surveys	Google Forms	20 students	2 weeks	Assess safety perception, incident experience, and app adoption intent.
Direct observation	Record form	4 campus zones	1 week	Record activity level and suspicious events by time and zone.
Document analysis	Institutional reports	Campus records	3 days	Identify historical incident patterns and cross-validate field findings.
Benchmarking	Comparative analysis	3 apps in Bogotá	1 week	Extract design insights from existing platforms.
Weather monitoring	Weather apps	Tousaquillo locality	1 week	Explore correlation between adverse weather and incident occurrence.

6 | SYSTEM ARCHITECTURE



The CSAS architecture consists of five principal modules:

User Interface Layer

Responsible for incident reporting and alert visualization.

Incident Service

Receives and stores reports.

Verification Service

Evaluates reports using confidence-based validation mechanisms.

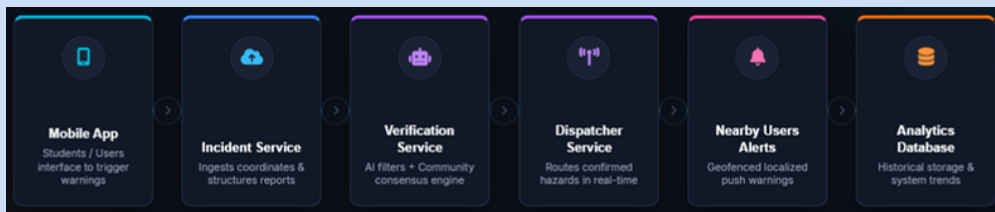
Dispatcher Service

Broadcasts verified alerts to nearby users.

Analytics Service

Stores historical information and generates statistical reports.

The modular architecture promotes maintainability and future scalability.



7 | CONCLUSIONS

The Community Security Alert System successfully integrates requirements engineering, system design principles, and simulation-based validation into a coherent security solution.

Survey data, direct observations, and benchmarking activities confirmed the need for improved reporting and notification mechanisms within university campuses.

The resulting architecture addresses these challenges through a collaborative, real-time, and data-driven approach.

Simulation results demonstrated that the system can satisfy critical functional and non-functional requirements while maintaining acceptable performance levels.

Consequently, CSAS represents a viable foundation for future implementation as a fully operational campus security platform.